Pthreads

definition

established in 1995 under the IEEE POSIX 1003.1c standard, this defines a specific API (Application Programming Interface) for managing threads

it is fundamentally a C language interface
- it provides a set of programming types and procedure calls that allow you to manipulate threads directly
- from a practical standpoint, if you are coding in C/C++ on a Linux system
  - you must include the header file <pthread.h>
  - when compiling, you almost always need to link the library using the -phtread flag in GCC

structure of the API

the Pthreads API is quite extensive, but conceptually, we can divide its functionality into two main categories

thread management
- administrative functions that handle the lifecycle of a thread
  - **creation:** spawning a new thread
  - **joining:** waiting for a thread to finish (synchronizing the end of execution)
  - **detaching:** telling the OS to clean up a thread automatically when it finishes
  - **attributes:** setting or querying specific properties of a thread

synchronization
- Pthreads provied some tools to coordinate access to shared memory as
  - **mutexes (mutual exclusion):** these act as locks to protect critical sections of code
  - **condition variables:** these allow threads to communicate explicitly, letting one thread sleep until another signals that a specific condition has been met

thread management

creation

once threads are created they are considered **peers**
- unlike processes which often have a strict parent-child hierarchy, threads in Pthreads effectively exist at the same level
  - any thread can create another thread
  - there is no implied dependency hierarchy between them

qualsiasi thread può creare altri thread o terminare altri thread → tutti i thread sono un "pari" del creatore

- however, resources are not infinite
  - the maximum number of threads you can spawn depend heavily on the specific implementation and the operating system's limits

- il nuovo thread ottiene il proprio stack privato (per le variabili locali), i propri registri e il proprio Program Counter
- condivide con il creatore (e gli altri thread): il codice, le variabili globali (Data segment), l'Heap (memoria allocata con malloc), i file aperti e il PID

- this model is slightly more "coarse-grained" than CUDA
  - OpenMP often manages thread pools for you, but in Pthreads, you manage creation explicitly
  - dobbiamo gestire più livelli prima di far partire l'esecuzione vera

to actually spawn a thread, we use the pthread_create subroutine

```
int pthread_create(pthread_t * thread,
                   const pthread_attr_t * attr,
                   void * (* start_routine) (void *),
                   void *arg)
```

questa funzione chiede al sistema operativo di allocare le risorse per un nuovo flusso di esecuzione (stack, registri, program counter) e di avviarlo

- output ID — pthread_t * **thread**
  - **thread identifier:** a pointer to a pthread_t variable where the system will store the unique ID of the new thread
  - puntatore a una variabile di tipo pthread_t → la funzione scriverà in questa variabile l'ID univoco del nuovo thread creato
- puntatore a una funzione che definisce le proprietà del thread — attributi — const pthread_attr_t * **attr**
  - **attributes:** this allows you to customize the thread, such as setting its stack size or determining if it is joinable or detached
    - passing NULL usually accepts the default settings
      - si può personalizzare inizializzando la struttura con pthread_attr_int
  - joinable/detached
    - riguarda il managing del sistema
  - scheduling
    - riguarda le caratteristiche che lo user si aspetta da quel thread
      - questo attributo controlla come il sistema operativo decide quando far eseguire il suo thread rispetto agli altri processi o thread nel sistema
  - stack size
    - l'amount di stack che dobbiamo riservare a quel thread
      - ogni thread ha il proprio stack privato, separato da quello degli altri thread, dove vengono memorizzate le variabili locali e le chiamate a funzione
      - aumentarlo → se il thread fa molte chiamate ricorsive o alloca array locali molto grandi, potremmo aver bisogno di più spazi rispetto al default per evitare un *stack overflow*
      - diminuirlo → se devi creare migliaia di thread che fanno pochissimo lavoro, ridurre lo stack size ti permette di risparmiare memoria globale
- "main" del thread — void * (* **start_routine**) (void *)
  - **start routine:** it is a pointer to the C function that the thread will execute
    - indirizzo della funzione a cui andremo ad associare il nostro thread
  - this function must have a specific signature
    - it returns a void *
      - prende un puntatore generico e ritorna un puntatore generico
    - takes a void * as an argument
- argomento — void ***arg**
  - this is the **argument** passed to that start routine
    - because it is a void pointer, you can pass the address of any data structure. allowing you to pass complex data to the thread

termination

once a thread is running, there are a few ways a thread's life ends

la sua funzione start routine termina e restituisce un void

- **natural completion:** the thread simply returns from its starting routine
- **explicit exit:** the thread calls pthread_exit
  - useful if you want to terminate the thread in the middle of the routine
    - explicitly pass a return value back to the creator
      - si può utilizzare per passare dei valori all'utente che sta effettuando una chiamata diversa da quella dello user
        - NB se dobbiamo passare più parametri, dobbiamo creare una struct, riempirla con i dati e passare l'indirizzo di quella struct
  - `void pthread_exit(void *retval);`
    - il parametro retval è un puntatore void* che rappresenta il risultato del thread → viene reso disponibile al thread che farà la join su questo thread
- **cancellation:** another thread forces it to stop using pthread_cancel — `int pthread_cancel(pthread_t thread);` — invia una richiesta di cancellazione al thread specificato dall'ID thread
- **process termination:** if the main process terminates, *all* thread are immediately killed, regardless of their state
  - therefore, if you want thread to survive, the main program must typically call pthread_exit as the last thing it does
    - rather than just returning or falling off the end of main

NB if the main thread calls pthread_exit() instead of returning, it will terminate itself but allow other threads to continue executing

Il Caso Speciale del `main`: `return` vs `pthread_exit`

Questa è una distinzione cruciale per l'esame.

- **Se il `main` fa `return` (o finisce):** Il sistema operativo chiude l'intero processo. Tutti i thread secondari vengono uccisi immediatamente, anche se non hanno finito il loro lavoro.
- **Se il `main` chiama `pthread_exit`:** Il thread principale muore, **MA il processo rimane vivo** se ci sono altri thread ancora in esecuzione. Questo permette ai thread figli di completare il loro lavoro anche se il "padre" (il main) ha finito. :P

  Esempio pratico: Se lanci dei worker e non vuoi fare la `join` su tutti, puoi chiamare `pthread_exit(NULL)` alla fine del `main`. Il programma terminerà solo quando l'ultimo worker avrà finito.

joining

in a typical manager/worker model, the main thread spawns workers and then need to wait for them to finish

joining è il meccanismo che permette a un thread di sospendere la propria esecuzione e aspettare che un altro specifico thread termini il suo lavoro

`int pthread_join(pthread_t thread, void ** retval)`

- when we call pthread_join on a specific thread ID, our current thread stops and waits until that target thread terminates
  - pthread_t thread è l'ID del thread che vogliamo aspettare → quello che avevamo ottenuto dalla pthread_create precedente
- the second argument to pthread_join allows you to capture the return value (the retval) that the target thread passed to pthread_exit
  - this function also serves as a data retrieval mechanism
- pthread_join permette di sincronizzare l'esecuzione tra il master thread e i subthreads con l'ordine di esecuzione che dipende solo da OS e dalle risorse usate
  - lavora come una sorta di synchronization primitive
  - possiamo chiedere all'OS di creare altri thread che continuano a lavorare e che al termine delle oeprazioni vanno a chiamare pthread_exit()
    - modo per comunicare con il master thread che si è completato il processo

se il thread target aveva restituito un valore (tramite return o pthread_exit), quel valore (un puntatore void*) viene copiato nella variabile puntata retval → possiamo passare semplicemente NULL se non abbiamo bisogno del valore di ritorno

state of a thread

- **joinable**
  - a joinable thread keeps its resources (like its stack and return value) allocated even after it finishes, wating for someone to call pthread_join on it
    - if you don't join it, you might leak resources (create a "zombie" thread)

non è possibile recuperare il valore di ritorno

- **detached**
  - if you know you will never need to join a thread, you can create it as detached or detach it later
    - when a detached thread finishes, its resources are released immediately back to the system
  - this is a one-way street
    - a joinable thread can be detached
    - a detached thread cannot be made joinable again
- we have to explicitly set joinable or detached attribute
  - default attribute should be joinable
  - not all implementations adopt this default
  - detached threads may consume less resources

joining through barriers

- pthread_join waits for a thread to *terminate*
  - however, in many parallel algorithms, we need threads to synchronize *during* execution, not just at the end

mentre pthread_join è una sincronizzazione "uno-a-uno", la barriera è una sincronizzazione "molti-a-molti"

- we use **barriers** that acts as a checkpoint
  - we initialize it with a specific **count** (the number of threads involved)
    - `int pthread_barrier_init(pthread_barrier_t * barrier, pthread_barrierattr_t * attr, unsigned int count)`
    - `int pthread_barrier_wait(pthread_barrier_t * barrier)` — puntatore alla barriera su cui ci si vuole sincronizzare

I Parametri:

1. `pthread_barrier_t *barrier`: È un puntatore alla variabile barriera che hai dichiarato. Serve per identificarla.
2. `const pthread_barrierattr_t *attr`: Specifica gli attributi della barriera.
   - Per l'esame: Passa quasi sempre `NULL` per usare i comportamenti di default.
3. `unsigned int count` **(CRUCIALE):** Questo è il numero **esatto** di thread che devono chiamare la funzione di wait prima che la barriera si sblocchi.
   - *Attenzione:* Se imposti `count` a 5 e solo 4 thread chiamano la wait, il programma si bloccherà per sempre (Deadlock).

  - as threads reach a specific point in their code, they call pthread_barrier_wait
    - if a thread calls this and the count hasn't been reached, it blocks (stops executing)
      - se il thread corrente NON è l'ultimo (ossia numero di thread < count), il thread si blocca e dorme
    - once the last required thread (the N-th thread) calls the function, the barrier "breaks" and all threads resume execution simultaneously
      - se il thread corrente È l'ultimo (ossia numero di thread = count), la barriera sblocca tutti i thread in attesa e tutti riprendono l'esecuzione
- this is crucial for staged algorithms where Step 2 cannot begin until every thread has finished Step 1
  - barriers are an optional part of the POSIX standard, so not every implementation includes them

mutexes (mutual exclusion)

- basic method to protect shared data when multiple writes occur
  - think of a mutex as a lock on a door leading to a critical section of code (where shared data is modified)
    - only one thread can hold the lock at any given time

```
pthread_mutex_lock(&my_lock);
/* critical section */
pthread_mutex_unlock(&my_lock);
```

- to enter the room, you call pthread mutex_lock
  - if the lock is open, you enter and close it
    - se il mutex è libero, lo blocca, diventa il proprietario e la funzione ritorna immediatamente
  - if it is already locked by another thread, you get blocked and must wait until it opens
    - se il mutex è già bloccato, il thread chiamante viene sospeso e messo in una coda di attesa → si sveglierà solo quanto il mutex verrà sbloccato
- when you are done, you must call pthread_mutex_unlock to let others in
  - rende il mutex disponibili → se c'erano altri thread in attesa, uno di essi viene svegliato e acquisisce il mutex

è obbligatorio da chiamare alla fine della sezione critica

- risk: **deadlock**
  - example
    - imagine a scenario with two locks, A and B
      - Thread 1 locks A and then tries to lock B
      - Thread 2 locks B and then tries to lock A
    - if they execute these sequences simultaneously, Thread 1 waits for B (held by Thread 2) and Thread 2 waits for A (held by Thread 1)
      - they will wait forever
  - to help manage performance and logic, Pthreads provides variants
    - trylock
      - this attempts to lock the mutex
        - serve per "rinunciare" se il secondo lock non è disponibile, rilasciare il primo e riprovare più tardi
      - if it's busy, instead of blocking (sleeping), it returns an error immediately, allowing the thread to go to do other work
    - **recursive mutexes**
      - these allow a thread to lock the same mutex multiple times without deadlocking itself

synchronization

- while mutexes protect *data*, condition variables allow thread to synchronize based on the *state* of that data
  - without condition variables, if a thread needs to wiat for a condition to become true (e.g. "is the queue emptu?"), it would have to loop and check the variable constantly (polling)
    - this wastes CPU cycles

condition variables

meccanismo di sincronizzazione che permette ai thread di sospendere l'esecuzione e attendere che si verifichi una specifica condizione sui dati condivisi

- condition variables solve this by allowing a thread to sleep until another thread signals that the condition has changed
  - **waiting**
    - `int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)`
      - questa funzione mette il thread chiamante in attesa finché non riceve un segnale
        - il thread deve aver già acquisito il mutex prima di chiamare questa funzione
        - quando viene chiamata pthread_cond_wait, il sistema rilascia atomicamente il mutex e emette il thread a dormire
          - permette ad altri thread di modificare la memoria condivisa (e quindi rendere vera la condizione attesa)
        - quando il thread viene svegliato (da un segnale), la funzione tenta automaticamente di riacquisire il mutex prima di restituire il controllo al codice
    - a thread calls wait on a condition variable
      - it must hold a mutex before doing this
      - the system atomically unlocks the mutex and puts the thread to sleep
  - **signaling**
    - another thread, after changing the state (e.g., adding an item to the queue), calls signal to wake up the sleeping thread
    - `int pthread_cond_signal(pthread_cond_t *cond)`
      - sblocca (sveglia) un solo thread che è bloccato sulla condition variable specificata
        - si usa quando l'evento che si è verificato, può essere gestito da un solo thread
        - si usa quando l'evento che si è verificato può essere gestito da un solo thread (ad esempio, è stata liberata una singola risorsa in una coda)
          - se nessun thread è in attesa, il segnale viene perso (non viene memorizzato)
    - `int pthread_cond_broadcast(pthread_cond_t *cond)`
      - sveglia tutti i thread che sono attualmente bloccati su quella condition variabile
        - utile quando il cambiamento di stato deve essere notificato a tutti i consumatori

- this mechanism allows for efficient, explicit coordination between threads without busy waiting